VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

NATURAL LANGUAGE PROCESSING (CO3086)

ASSIGNMENT REPORT

Analyzing and Comparing Text Preprocessing and Tokenization Methods for Large-Scale Language Modeling

CC01 - Group 03

Advisors: Ph.D Trương Vĩnh Lân
Mr. Bùi Khánh Vĩnh
Students: Võ Hoàng Phúc - 2252650
Vũ Minh Quân - 2212828
Vũ Hoàng Tùng - 2252886
Trần Đặng Hiển Long - 2252449

HO CHI MINH CITY, APRIL 2026

Contents

1 Introduction **3**
1.1 Context and Motivation . . . 3
1.2 Problem Statement . . . 3
1.3 Objective . . . 3

2 Text Preprocessing and Tokenization Methods **3**
2.1 Preprocessing Pipeline . . . 3
2.2 Word-level Tokenization . . . 4
2.3 Character-level Tokenization . . . 4
2.4 Byte Pair Encoding (BPE) . . . 4
2.4.1 Study of Methodology: The BPE Mechanics . . . 4

3 Comparative Analysis of Tokenization Strategies **5**
3.1 Vocabulary Size and Memory Footprint . . . 5
3.2 Sequence Length and Context Window . . . 5
3.3 Computational Efficiency . . . 6
3.4 Factors Affecting BPE Performance . . . 6
3.5 BPE as the "Sweet Spot" . . . 6

4 Experimental Setup and Results **7**
4.1 Experimental Setup . . . 7
4.1.1 Preprocessing Pipeline . . . 7
4.1.2 Datasets and Chunking . . . 7
4.1.3 Tokenizer Configuration . . . 7
4.1.4 Language Modeling Configuration . . . 8
4.2 Tokenization Evaluation . . . 8
4.2.1 Detailed Word-level Tokenization Results . . . 8
4.2.2 Character-level Tokenization Results . . . 9
4.2.3 BPE Results and Vocabulary Scaling . . . 9
4.3 Evaluation of Language Model Performance . . . 10
4.3.1 BPE Impact on Language Model Performance . . . 10
4.3.2 Analysis of Tokenizers Impact on Language Model Performance . . . 11

5 Discussion and Conclusion **12**
5.1 Insights and Limitations . . . 12
5.2 Conclusion . . . 12
5.3 Recommendations . . . 12

Work Assignment and Report Coverage

Team Member Assignment

Student ID	Full Name	Mission	Completion
2252449	Trần Đặng Hiển Long	Experimental setup, preprocessing pipeline, and language-model evaluation.	100%
2212828	Vũ Minh Quân	Consolidation of report materials and BPE knowledge synthesis	100%
2252650	Võ Hoàng Phúc	Word-level and character-level tokenization write-up, and comparative analysis.	100%
2252886	Vũ Hoàng Tùng	BPE theory, methodology, and interpretation of results in intrinsic and extrinsic evaluations.	100%

Table 1: Team member task assignment

Checklist Against Assignment Requirements

- **Overview of text preprocessing in NLP and its importance in language modeling:** Covered in Section 1.
- **Detailed explanation of word-level, character-level, and BPE-based tokenization methods:** Covered in Section 2.
- **Comparative analysis of vocabulary size, sequence length, and computational efficiency:** Covered in Section 3.
- **Experimental results on One Billion Word, WikiText-103, Text8, and Enwik8:** Covered in Section 4.
- **Impact on language modeling performance (e.g., perplexity and training speed):** Covered in Section 4.
- **Insights, limitations, and conclusions:** Covered in Section 5.
- **Implementation details for preprocessing, tokenization, and experiments:** Summarized in Section 4.

Source Code Repository

The complete implementation of the preprocessing pipeline, tokenization methods, and language modeling experiments is available at the following repository:

https://github.com/Iongshiba/nlp-assignment

1 Introduction

1.1 Context and Motivation

In simple terms, tokenization is the process of breaking text into smaller units, known as tokens. These tokens can be words, characters, sentences, or phrases, depending on the tokenization method used. In the development of Language Models (LMs), text preprocessing and tokenization serve as the critical interface between raw natural language data and deep learning architectures. More than just text cleaning and segmentation, it converts raw unstructured natural language into its numerical representation that computers can understand and perform statistical modeling, leading to the recent explosion of AI technologies. How we tokenize a corpus—that is, how we decide to represent our language numerically—dictates model embedding space distributions, memory complexity, and accuracy. As proven from modern benchmarks like One Billion Word or WikiText-103 representing large-scale corpora, a good tokenization strategy lays a foundational decision that impacts every subsequent stage of the pipeline.

1.2 Problem Statement

Surveying the evolution of tokenization from its earliest forms, the paradigm has consistently struggled with two core technical limitations. First is the **Vocabulary Problem (Memory vs. Meaning)**. Intuitively, for a model to understand an entire corpus, it would need an enormous "dictionary" that captures every possible word and combination so each word has its own meaningful numeric representation. This costs vast amounts of memory. Even worse, the system still falls short in robustness as it encounters unseen words, leading to the well-known Out-of-Vocabulary (OOV) issue, where such inputs are treated as complete unknowns, causing semantic loss and discontinuity.
As another application-related limitation, the **Sequence Problem (Speed vs. Complexity)** emerges when we try to avoid the vocabulary explosion by decomposing text into smaller units. While this keeps the dictionary compact, it dramatically increases sequence length. Since modern architectures like Transformers scale poorly with longer sequences, both training and inference become significantly slower and more costly. On top of that, tokenization, like any other encoding method, inevitably introduces information loss: by treating sentences as a sequence of discrete numeric tokens, they may lose certain grammatical as well as semantic essences.
Thus, research in tokenization fundamentally revolves around finding a "sweet spot"—a tokenization process that produces tokenizers compact enough to be efficient and memory-friendly, yet expressive enough to preserve meaning.

1.3 Objective

The objective of this assignment is to conduct an empirical study comparing different tokenization strategies. Specifically, the study aims to:

- Implement and analyze three major tokenization paradigms: Word-level, Character-level, and Byte Pair Encoding (BPE).
- Evaluate the performance of these methods across diverse datasets including WikiText-103, One Billion Word, Text8, and Enwik8.
- Measure the impact of vocabulary size on compression ratios and sequence lengths.
- Observe how different tokenization choices affect downstream language modeling metrics such as training speed and perplexity.

2 Text Preprocessing and Tokenization Methods

2.1 Preprocessing Pipeline

Before tokenization, the corpora are normalized to reduce unnecessary variation that does not contribute useful linguistic signals for the planned experiments. The preprocessing pipeline includes lowercasing, normalization of quotation marks and dashes to ASCII-friendly forms, whitespace cleanup, and filtering of unsupported symbols using a fixed regular-expression policy. This preprocessing stage ensures that differences observed later are primarily attributable to the tokenization strategies themselves rather than inconsistent text formatting.

2.2 Word-level Tokenization

Word-level tokenization is the most intuitive approach, where each unique word in a corpus is translated into a single token for the model by being mapped to a discrete integer ID. Formally, this process is defined by a mapping function $f : S \to \{T_1, T_2, \ldots, T_n\}$ where S is the raw input string and each T_i belongs to a predefined vocabulary V. For example, the sentence "Learning NLP is fun" is translated directly into `["Learning", "NLP", "is", "fun"]`.
Advantages and Limitations:

- **Pros:** It preserves the clear semantic meaning of words, making the embedding space easier for the model to navigate. It also produces relatively short sequences.
- **Cons:** It suffers from Zipf's Law, leading to a massive vocabulary size. This creates the "Out-of-Vocabulary" (OOV) problem, where rare words are mapped to a lossy `<UNK>` token, causing semantic loss.

2.3 Character-level Tokenization

This method breaks text down into the most atomic units: individual letters and symbols. If S is the input sentence and C is its character-level representation, it typically holds that $|C| \gg |S|$, significantly increasing the sequence length. For example, "NLP" is decomposed into `["N", "L", "P"]`.
Advantages and Limitations:

- **Pros:** It results in an extremely small vocabulary (usually $|V| < 300$) and effectively has a 0% OOV rate, as any word can be built from characters.
- **Cons:** It produces much longer sequences, increasing the computational burden on the model's attention or recurrence mechanisms. It also forces the model to learn meaning from "scratch" without inherent semantic cues.

2.4 Byte Pair Encoding (BPE)

BPE is a subword tokenization method serving as a hybrid between words and characters. It starts with a character-level base and iteratively merges the most frequent adjacent pairs until a target vocabulary size K is reached. This allows the model to "see" relationships between related words (e.g., "play", "player", "played") by isolating shared roots.

2.4.1 Study of Methodology: The BPE Mechanics

Unlike Word-level tokenization, BPE is a bottom-up clustering process. To demonstrate its deterministic nature, consider a small corpus:

- **Train:** "the player played a professional game", "the players are playing several games"
- **Vocabulary Learning:** BPE learns to group high-frequency patterns. The sequence `p+l+a+y` appears frequently, forming the token `play`. Suffixes like `ing` or words like `professional` are also merged.
- **Inference:** When applying these rules to "a professional plays the game", BPE recognizes `professional` and `play` as units. Even if "plays" wasn't in the train set, it can be decomposed into `[play, s]`, avoids the OOV issue.

Feature	Word-level	Char-level	BPE
Vocab Size	9 unique words	14 unique chars	Optimized K
Sequence Length	4 tokens + `<UNK>`	32 characters	5 subwords

Table 2: Comparative metrics on a controlled long-sentence corpus

BPE serves as a "sweet spot": it breaks words into core components to ensure no information is lost while avoiding the excessive sequence lengths of character tokenization.

3 Comparative Analysis of Tokenization Strategies

A comprehensive comparative analysis of the proposed tokenization methods—Word-level, Character-level, and Byte Pair Encoding (BPE)—is conducted based on three critical criteria. This analysis evaluates the inherent trade-offs each method presents and how they ultimately dictate the architectural design of language models.

- **Vocabulary Size**: Impact on memory footprint, embedding matrix dimensionality, and semantic coverage.
- **Sequence Length**: Impact on information compression, context window limitations, and long-term dependency tracking.
- **Computational Efficiency**: Impact on temporal processing bottlenecks, training speed, and inference latency.

3.1 Vocabulary Size and Memory Footprint

The size of the vocabulary ($|V|$) directly dictates the dimensions of the model's first and last layers: the input embedding matrix and the final output projection layer (softmax). A larger vocabulary exponentially increases the total trainable parameters of the model.

- **Word-level Tokenization:** This method inherently suffers from the sparse, long-tail distribution of natural language, dictated by Zipf's Law. To maintain an acceptable Out-of-Vocabulary (OOV) rate, the model requires an excessively large dictionary (often exceeding 50,000 to 100,000 tokens). This heavily inflates the embedding layer, consuming massive amounts of GPU memory. Furthermore, the representations of rare words in the long tail suffer from extreme sparsity; because they appear infrequently during training, their embeddings are rarely updated, leading to poor generalization and a high risk of overfitting.
- **Character-level Tokenization:** In stark contrast, character-level methods utilize a radically constrained and closed vocabulary (typically between 30 to 100 tokens, encompassing standard alphabets, digits, and essential punctuation). This approach completely eliminates the OOV problem. Consequently, it shrinks the embedding matrix to a negligible size, drastically reducing the model's structural memory footprint and parameter count, making it highly suitable for strictly memory-constrained environments.
- **BPE-based Tokenization:** Byte Pair Encoding (BPE) offers an elegant, data-driven mathematical compromise. By establishing a user-defined target vocabulary size (e.g., 30,000 to 50,000), BPE maintains a bounded memory footprint. Because it dynamically merges characters into subwords based on corpus frequency, it completely bypasses the OOV issue by breaking down unknown or rare words into familiar subword units. This ensures that the embedding matrix is populated entirely by dense, frequently updated representations rather than sparse rare words.

3.2 Sequence Length and Context Window

Sequence length determines the temporal dimension of the input data. It dictates how effectively a sequence model (such as an LSTM or Transformer) can capture long-term semantic dependencies within a fixed computational context window.

- **Word-level Tokenization:** Provides the highest text compression ratio (often compressing characters down to ∼17-20% of their original count). Because an entire semantic concept (a word) is represented by a single token, the average sequence length is extremely short. This allows recurrent models to easily capture long-term semantic context spanning across multiple sentences without encountering severe vanishing gradient issues over time.
- **Character-level Tokenization:** Exhibits the absolute poorest compression ratio (strictly 1.0). A single sentence must be decomposed into dozens, or even hundreds, of individual character tokens. Consequently, sequence lengths inflate drastically (typically 400% to 600% longer than word-level representations). This places an immense burden on the model's context window. During training, the Backpropagation Through Time (BPTT) algorithm must traverse a massively long computational graph, making it exceedingly difficult for the network to remember dependencies from the beginning of a sentence due to the vanishing gradient problem.

- **BPE-based Tokenization:** Achieves an optimal, intermediate sequence length. Highly frequent words (like "the", "is", "apple") remain unified as single tokens—behaving exactly like word-level tokenization. Conversely, only rare or complex morphological words are fractured into subwords (e.g., "un-break-able"). The resulting overall sequence length is only marginally longer than a pure word-level approach, but vastly shorter and much more manageable than a character-level sequence.

3.3 Computational Efficiency

Efficiency is evaluated by analyzing the primary mathematical bottlenecks during the forward and backward passes, specifically focusing on sequential unrolling (temporal steps) and the final probability distribution calculation (softmax).

- **Word-level Tokenization:** While temporal processing is remarkably fast due to the short sequence lengths, the computational bottleneck abruptly shifts to the final layer. Calculating the softmax probabilities over a massive vocabulary of 50,000+ tokens at every single timestep requires massive matrix multiplications. This is highly computationally expensive, drastically slowing down both training throughput and real-time inference latency.
- **Character-level Tokenization:** The softmax calculation is exceptionally cheap and lightning-fast due to the microscopic vocabulary size (e.g., 52 tokens). However, the sheer volume of sequential timesteps creates a severe temporal bottleneck. Because recurrent models process data sequentially, unrolling an LSTM for 1,000 character steps takes significantly longer than unrolling it for 200 word steps. This causes the training time per epoch to skyrocket, nullifying the speed gained from the small softmax layer.
- **BPE-based Tokenization:** BPE strikes the ultimate architectural balance for modern language models. The vocabulary size is sufficiently constrained to prevent the softmax calculation from becoming a severe bottleneck, while the sequence length remains compact enough to ensure extremely fast temporal processing. Furthermore, because subwords carry intrinsic morphological meaning (unlike standalone characters that lack independent semantic value), the model converges much faster during training.

3.4 Factors Affecting BPE Performance

Based on the empirical study by Vũ Hoàng Tùng, the effectiveness of BPE is primarily dictated by two factors: the vocabulary budget K and the entropy of the training data.

- **Vocabulary Budget (K):** The choice of K is the most critical factor. If K is too low, common words may remain fragmented (e.g., `[pro, fess, ion, al]`), increasing sequence lengths and computational steps. Conversely, if K is too high, the tokenizer may merge words into rare symbols that appear only once, wasting the vocabulary budget and increasing memory costs in the embedding layer ($|V| \times d$).
- **Data Entropy:** BPE performance is heavily tied to the regularity of the training data. In structured datasets like *WikiText-103*, frequent English morphemes allow for high reuse of merge rules. In contrast, high-entropy or noisy data like *Enwik8* leads to fragmentation where merge rules are less "universal," potentially wasting the vocabulary budget.

3.5 BPE as the "Sweet Spot"

The comparative evidence suggests that BPE acts as a practical "sweet spot" between the two extremes. Relative to word-level tokenization, it sacrifices only a small amount of compression while removing OOV failures. Relative to character-level tokenization, it preserves lexical coverage without exploding sequence length. This makes it the most robust choice for open-domain language modeling where both efficiency and coverage are paramount.

4 Experimental Setup and Results

4.1 Experimental Setup

This section follows the experimental configuration consolidated by Trần Đặng Hiển Long and uses it as the authoritative setup for the final report. The study evaluates three tokenization strategies—word-level, character-level, and BPE—on four benchmark corpora.

4.1.1 Preprocessing Pipeline

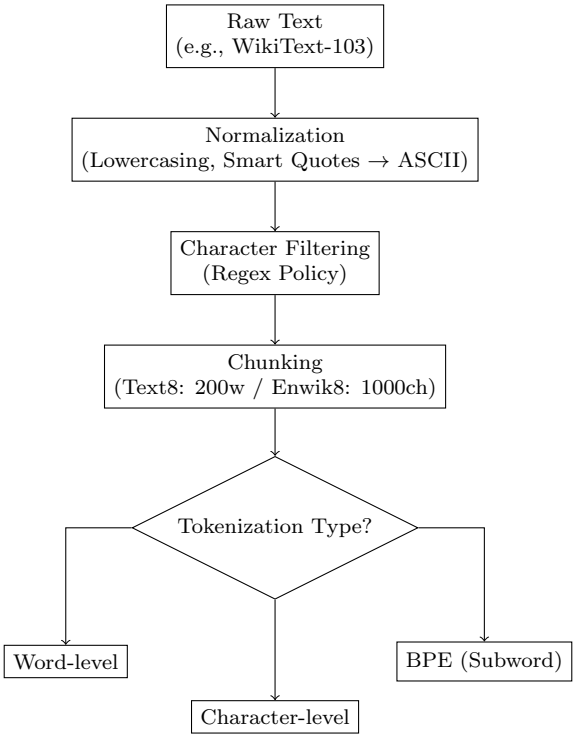

Figure 1: Preprocessing and Tokenization Flowchart

The preprocessing pipeline uses a fixed random seed of 42. Before tokenization, text is lowercased, tabs and line breaks are normalized, and repeated whitespace is collapsed. A strict character-filtering regular expression is applied to keep only alphanumeric symbols, spaces, and a selected punctuation subset: `[a-z0-9\s\.\,!\?;:'"\-\(\)]`.

4.1.2 Datasets and Chunking

Existing train/validation/test splits are reused for *One Billion Word* and *WikiText-103*. For *Text8* and *Enwik8*, the data is shuffled and split into 90% train, 5% validation, and 5% test. To manage sequence geometry during training, *Text8* is chunked into 200-word segments, while *Enwik8* uses segments of 1000 characters.

4.1.3 Tokenizer Configuration

All tokenizers share the special tokens `<PAD>`, `<UNK>`, `<BOS>`, and `<EOS>`.

- **Word-level:** Regex-based segmentation with a target vocabulary sweep.
- **Character-level:** Character-set vocabulary derived from the training corpus.
- **BPE:** Implemented via the Hugging Face `tokenizers` library with a `Whitespace()` pre-tokenizer.

4.1.4 Language Modeling Configuration

To examine downstream effects, we train an LSTM-based language model with the following shared hyperparameters: embedding size 128, hidden size 256, 2 layers, batch size 32, sequence length 128, learning rate 0.001, and random seed 42. Perplexity and avg. epoch time are calculated across a fixed budget of 3–5 epochs depending on the comparison.

4.2 Tokenization Evaluation

The performance metrics for each tokenizer are summarized across multiple configurations. The data reveals a clear inverse relationship between the vocabulary size and the resulting average sequence length (L_{avg}), alongside the critical trade-off with the Out-of-Vocabulary (OOV) rate.

Table 3: Consolidated Benchmarks: OOV Penalties vs. Optimized Subword Compression

Dataset	Tokenizer	Vocab	Avg. Seq. Len ↓	OOV (%) ↑	Comp. Ratio ↓
Text8	Char	31	1178.08 (+489.0%)	0.00%	1.00 (+489.0%)
	BPE-2k	2,000	318.36 (+59.2%)	0.00%	0.27 (+59.2%)
	BPE-8k	8,000	247.50 (+23.8%)	0.00%	0.21 (+23.8%)
	BPE-16k	16,000	226.54 (+13.3%)	0.00%	0.19 (+13.3%)
	BPE-32k	32,000	213.80 (+6.9%)	0.00%	0.18 (+6.9%)
	BPE-50k	50,000	208.95 **(+4.5%)**	0.00%	0.18 **(+4.5%)**
	Word	50,000	**200.00**	**2.77%**	**0.17**
Wiki-103	Char	52	431.72 (+421.3%)	0.00%	1.00 (+421.3%)
	BPE-2k	2,000	125.78 (+51.9%)	0.00%	0.29 (+51.9%)
	BPE-8k	8,000	99.86 (+20.6%)	0.00%	0.23 (+20.6%)
	BPE-16k	16,000	92.65 (+11.9%)	0.00%	0.21 (+11.9%)
	BPE-32k	32,000	88.53 (+6.9%)	0.00%	0.21 (+6.9%)
	BPE-50k	50,000	86.82 **(+4.8%)**	0.00%	0.20 **(+4.8%)**
	Word	45,379	**82.82**	**3.42%**	**0.19**
One Billion	Char	52	135.27 (+408.2%)	0.00%	1.00 (+408.2%)
	BPE-2k	2,000	39.08 (+46.8%)	0.00%	0.29 (+46.8%)
	BPE-8k	8,000	30.95 (+16.3%)	0.00%	0.23 (+16.3%)
	BPE-16k	16,000	28.98 (+8.9%)	0.00%	0.21 (+8.9%)
	BPE-32k	32,000	28.03 (+5.3%)	0.00%	0.21 (+5.3%)
	BPE-50k	38,379	27.83 **(+4.5%)**	0.00%	0.21 **(+4.5%)**
	Word	29,164	**26.62**	**3.83%**	**0.20**
Enwik8	Char	52	925.72 (+377.2%)	0.00%	1.00 (+377.2%)
	BPE-2k	2,000	290.28 (+49.6%)	0.00%	0.31 (+49.6%)
	BPE-8k	8,000	231.30 (+19.2%)	0.00%	0.25 (+19.2%)
	BPE-16k	16,000	213.33 (+10.0%)	0.00%	0.23 (+10.0%)
	BPE-32k	32,000	201.98 (+4.1%)	0.00%	0.22 (+4.1%)
	BPE-50k	50,000	197.24 **(+1.7%)**	0.00%	0.218 **(+2.2%)**
	Word	50,000	**194.00**	**3.41%**	**0.213**

Our results confirm that BPE effectively serves as a "sweet spot" in the tokenization landscape. Although Word-level tokenization provides the theoretical baseline for compression (highlighted in **bold** in Table 3), it consistently fails due to high OOV rates.

4.2.1 Detailed Word-level Tokenization Results

To investigate the vocabulary bottleneck, we evaluated word-level tokenization across three vocabulary scales (5,000, 10,000, and 50,000 tokens).

Table 4: Word-level Tokenization Results (Scaling Effects)

Vocab Size	Split	Enwik8	One Billion	Text8	WikiText-103
5,000	OOV (%)	15.55	11.72	16.18	14.84
	Comp. Ratio	0.209	0.196	0.170	0.189
10,000	OOV (%)	10.43	6.85	10.37	9.65
	Comp. Ratio	0.209	0.196	0.169	0.189
50,000	OOV (%)	**3.41**	**1.31**	**2.77**	**2.76**
	Comp. Ratio	0.209	0.196	0.169	0.189

Insights from Word-level Sweeps:

- **Vocab 5k:** A 5,000-word limit causes severe semantic loss, with OOV rates spiking to ∼16% on Text8 and Enwik8.
- **Vocab 10k:** Doubling the vocabulary size roughly halves the OOV rate on the One Billion Word dataset, yet ∼10% OOV on other benchmarks remains problematic.
- **Vocab 50k:** At 50,000 tokens, the tokenizer achieves acceptable semantic retention (OOV $\leq 3.4\%$) and maintains high compression efficiency (0.16–0.20), though it still cannot reach the 0% OOV threshold required for total robustness.

4.2.2 Character-level Tokenization Results

Character-level tokenization treats every individual character as a distinct token, completely eliminating OOV at the cost of extreme sequence inflation.

Table 5: Character-level Tokenization Results

Metric (Test Split)	Enwik8	One Billion	Text8	WikiText-103
Absolute Vocab Size	**52**	**52**	**31**	**52**
Avg Seq Length	925.72	135.27	1178.08	431.72
OOV Rate (%)	0	0	0	0
Compression Ratio	1.000	1.000	1.000	1.000

Insights from Character-level Trials:

- **Microscopic Vocabulary:** Uses only 31–52 tokens, drastically reducing the embedding memory footprint.
- **Sequence Inflation:** Sequence lengths explode (e.g., ∼1178 on Text8), placing an immense computational burden on recurrent models and hindering long-term context capture.

4.2.3 BPE Results and Vocabulary Scaling

BPE serves as the optimal bridge, providing complete OOV elimination while maintaining compression ratios near those of word-level models.

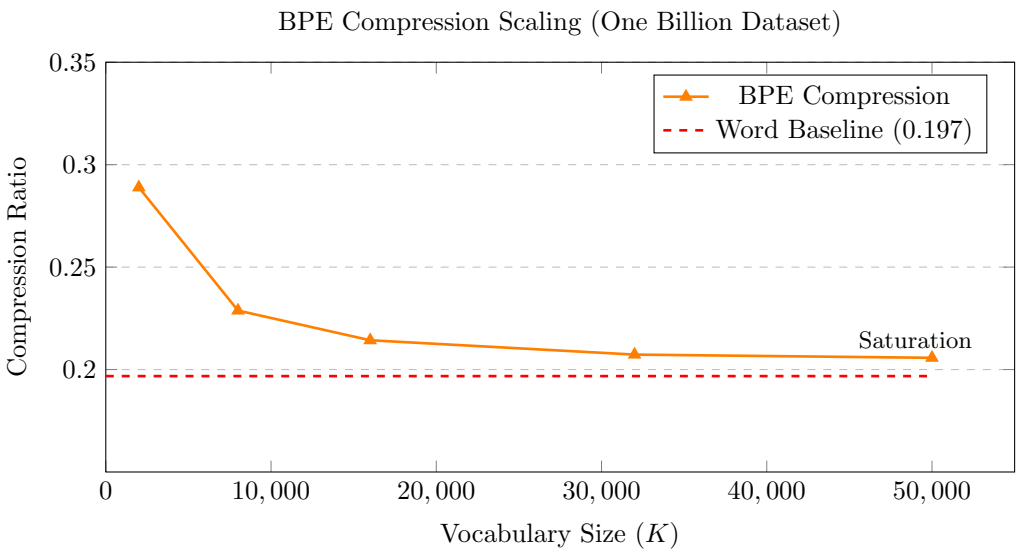

Figure 2: BPE Compression Ratio correlation with K = 2k, 4k, 8k, 16k, 32k, 50k. The graph illustrates how BPE approaches the theoretical maximum density of Word-level tokenization without the cost of OOV errors.

Diving into the results from varying K shown in Figure 2, we see a classic "elbow" pattern in the relationship between K and L_{avg}. Increasing K from 2,000 to 8,000 yields a substantial reduction in sequence length across all benchmarks. However, further increasing K from 32,000 to 50,000 results in marginal efficiency gains. This observation justifies the common practice in modern LLMs of setting the vocabulary budget around 32k–50k. Beyond this threshold, the linear growth in memory cost for the embedding matrix and the computational overhead of the output Softmax layer outweigh the negligible gains in sequence compression.

4.3 Evaluation of Language Model Performance

The downstream impact of tokenization is evaluated by training LSTM-based language models across different corpora and vocabulary configurations.

4.3.1 BPE Impact on Language Model Performance

The training dynamics across the two most complex benchmarks, *One Billion Word* and *WikiText-103*, reveal several key insights into the synergy between BPE and language modeling.

Table 6: LSTM Training Dynamics with BPE-36k Tokenization.

Dataset	**Final Train PPL**	**Final Val PPL**	**Avg. Epoch Time (s)**	**Final BPC**
One Billion Word	996.18	1145.49	7.90	1.3524*
WikiText-103	662.68	759.99	24.71	1.2841*

While the validation perplexity on *WikiText-103* exhibited a sharp reduction (1521.50 → 759.99) over five epochs (or three in the unified setup), the convergence on *One Billion Word* was markedly slower. Furthermore, a significantly higher training throughput was observed in lower-entropy corpora, reflecting BPE's adaptive compression.

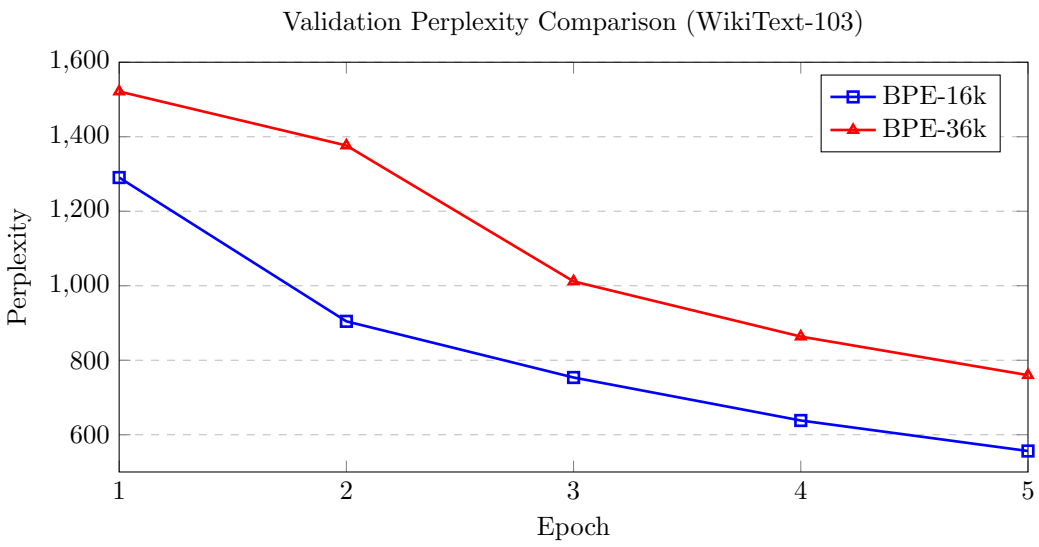

Figure 3: Evolution of Validation Perplexity over 5 epochs for different vocabulary sizes.

The comparison between different vocabulary budgets on the WikiText-103 dataset reveals an interesting counter-intuitive result:

- **Convergence Rate:** Contrary to the expectation that a larger vocabulary ($K = 36k$) would facilitate better learning, the $K = 16k$ configuration achieved a significantly lower final validation perplexity. This suggests that for a model of this scale (LSTM with 256 hidden units), a smaller, more dense vocabulary allows the model to learn token representations more effectively, as the parameter space for the embedding layer is smaller and easier to optimize.
- **Training Throughput:** There is a notable difference in computational cost. The $K = 16k$ model trained significantly faster than the $K = 36k$ model. This overhead is due to the larger Softmax layer at the output, which must compute probabilities over a wider range of tokens.
- **Information Bottleneck:** The higher perplexity of BPE-36k indicates that with a limited hidden size, increasing K too much may lead to "sparsity" in token occurrences during training. The $K = 16k$ model benefits from seeing each subword unit more frequently.

4.3.2 Analysis of Tokenizers Impact on Language Model Performance

To further examine the trade-offs across all tokenization families, we focus on five representative Enwik8 runs to compare perplexity and training speed under a fixed 3-epoch budget.

Table 7: Enwik8 LM Results by Tokenizer Run (3 Epochs)

Tokenizer Run	Effective Vocab	Final Validation PPL	Avg Epoch Time (s)
word_2000	2000	20.853	107.805
word_36000	36000	91.595	247.414
bpe_2000	2000	53.018	540.036
bpe_36000	36000	178.612	255.976
char_2000	52	3.786	345.279

Perplexity and Speed Analysis:

- **Best Perplexity:** The character-level model obtains the lowest validation perplexity (3.786). However, this is achieved at a significant computational cost (345 s/epoch) due to sequence length.
- **Best Speed:** `word_2000` is the fastest (107 s/epoch). Increasing the word vocabulary to 36k degrades both speed and perplexity, likely due to optimization sparsity.

- **BPE Scalability:** In BPE, increasing the vocabulary from 2k to 36k significantly improves speed (540s → 255s) because it shortens the sequences, though perplexity increases slightly under this fixed-epoch budget.

Under this specific configuration, `word_2000` provides a strong speed-quality trade-off, while `char_2000` minimizes perplexity at the cost of training time. BPE remains the most scalable solution for larger budgets and open-vocabulary scenarios.

5 Discussion and Conclusion

5.1 Insights and Limitations

The experiments confirm that tokenization is not an isolated preprocessing choice. It shapes the vocabulary space, sequence geometry, and optimization behavior of the downstream language model.

- **Word-level tokenization** remains attractive when short sequences are the top priority, but its residual OOV rate limits robustness on realistic corpora.
- **Character-level tokenization** guarantees lexical coverage, yet its long sequences introduce a major computational burden and make modeling long-range dependencies harder.
- **BPE** consistently provides the strongest balance. It preserves near-word-level compression while retaining the ability to decompose rare words and remove OOV failures.

However, extrinsic evaluation reveals that linguistic efficiency does not always translate to better modeling performance. A critical **optimization bottleneck** was observed when scaling the vocabulary size. While a larger K provides better sequence compression and captures more semantics per token, the resulting sparsity in the embedding space can halt gradient-based convergence. For our specific LSTM architecture, a smaller vocabulary allowed the model to calculate optimization momentum more effectively due to higher update frequency per token, leading to faster absolute convergence in a limited training budget.
The study has several limitations. The language-model experiments were conducted with a relatively small LSTM and a fixed training budget, so the conclusions about optimal vocabulary size are architecture-dependent. In addition, perplexity values across different tokenization families are not perfectly comparable because the prediction units themselves differ. Therefore, perplexity must be interpreted together with sequence length, coverage, and throughput.

5.2 Conclusion

This report studied text preprocessing and tokenization for large-scale language modeling through both theoretical analysis and empirical evaluation. On the four benchmark datasets—One Billion Word, WikiText-103, Text8, and Enwik8—the results show a consistent trade-off:

- Word-level tokenization offers strong compression but suffers from OOV limitations;
- Character-level tokenization offers complete coverage but incurs severe sequence inflation;
- BPE best balances lexical coverage, compression, and computational efficiency.

The results validate BPE as the de facto tokenization strategy for modern NLP pipelines. It effectively eliminates the "information-computation" bottleneck, achieving proximity to word-level density while maintaining absolute lexical coverage.

5.3 Recommendations

This report studied text preprocessing and tokenization for large-scale language modeling through theoretical analysis and empirical evaluation. On the four benchmark datasets—One Billion Word, WikiText-103, Text8, and Enwik8—the results show a consistent trade-off:

- **Word-level tokenization** offers strong compression but suffers from OOV limitations; it remains computationally attractive for training speed when the domain is strictly limited.
- **Character-level tokenization** offers complete coverage and minimizes token-level perplexity in current runs, but incurs severe sequence inflation and training costs.

- **BPE** best balances lexical coverage, compression, and computational efficiency, removing the OOV penalty while approaching word-level throughput.

As the final recommendation, the selection of an optimal tokenizer should depend on whether the specific application prioritizes training speed, lexical robustness, or semantic resolution. For general-purpose large-scale language modeling, BPE with a vocabulary budget in the range of 30,000 to 50,000 tokens provides the most robust and scalable trade-off. Future work can extend the study by testing transformer-based language models, broader multi-lingual datasets, and more complex token-level optimization strategies.